

## Report: Finnish to Rauma Finite-state Transducer (FST)

### Introduction

The Rauma dialect is a notoriously strong dialect of Finnish that is spoken in the southwestern coastal town of the same name. The port in the town exposed the language community to broader European influences and imported much of the dialect's unique vocabulary. It was also the first Finnish dialect to be standardized and published in a written form. Due to its early standardization, there is a small body of Rauma texts available on the internet that could be useful for natural language processing applications.

In a previous project, I attempted to create a Finnish-Rauma parallel corpus to train a neural machine translation (NMT) model. The NMT model gave decent results, but I suspected that the kinds of mutations between Rauma dialect and standard Finnish would be better suited for a more rigid algorithm like finite-state transducers (FST).

The goal of this project is to build such an FST that transforms a standard Finnish surface form into its Rauma dialect equivalent. This FST could then potentially be used to augment training data for a more robust NMT solution.

### Methodology

The final architecture of the project accepts as input a standard Finnish surface form which has been run through an off-the-shelf Finnish morphological analyzer from the Helsinki Finite-state Technology (HFST) group. The morphological tags outputted by the analyzer are then appended to the end of the surface form for additional context. Note that the lexical form outputted by the analyzer is effectively ignored. The benefit of this preliminary analysis is to disambiguate word forms that would otherwise be spelled the same way. See the example below for an overview of the preprocessing procedure for the surface form *tytön*:

$$\text{tytön} \rightarrow \text{tyttö+N+GEN} \rightarrow \text{tytön}^{\wedge}\text{+N+GEN}$$

In other words, the first step is the result of running *tytön* through the analyzer. The second step is the result of manually replacing the lexical form with the original surface form as well as an additional carrot for determining word boundaries later. The resulting form is what is fed into the Rauma generator that I have written.

The generator itself is an FST which was constructed entirely by writing rules in the Xerox Finite State Tool (XFST) language (see *rauma-generate.xfst*). The rules were sourced from a few different sources. The *Nortamo seor* (the Nortamo Society) which encourages the continued use of the Rauma dialect has published some resources for working with the language. On their website, they have a brief list of grammar rules. Their YouTube channel

has a short video series teaching these principles and a few additional ones. They have also published a dictionary called *Sillail oikke* which contains some grammar notes as well. I reviewed all of these resources and condensed them into a list of concrete rules (see *grammar.pdf*). I also added some of my own observations from reading Hjalmar Nortamo's short stories in Rauma.

The first section of the file is used for defining common components that could be used in the rules such as *V* for any vowel, *Nasal* for any nasal consonant, or *Syllable* for the Finnish syllable structure. The second section of the XFST file consists of all of the rules. The rules were further divided into vowel changes, consonant changes, verbs, adjectives, and cases and particles for easy organization. A final rule *RemoveAnnotation* removes the carrot and Finnish analyzer annotations from the end of the input. All of these rules are compiled together into a single regular expression and saved to an HFST file for easy use.

To help me in the development process, I created a test suite with at least two test cases per target transformation at the end of the XFST file. There were initially 24 target transformations (labeled A through X in the XFST file), but the initial tests for fast spoken forms were disregarded. Those could easily be resolved with a lookup table.

My workflow generally looked like choosing a rule to work on, writing it, and checking whether the test cases pass. That process would loop until I was satisfied with a rule, and then I would continue onto the next one. It was also good practice to check the other test cases to see if the rules had any unintended consequences that needed to be resolved. Sometimes those could be resolved by changing the order the rules were applied in. Other times the solution was more complicated.

## Experiments

A feature that I would have liked to successfully implement was lexical replacement. In other words, the idea was to use the raw output from the Finnish analyzer with the lexical form to be able to easily replace Finnish lexemes with their Rauma dialect translations. Once the lexeme was swapped out, the rest of the morphology would be generated on top. See the example below for an overview of the preprocessing procedure for the surface form *tytön* with lexical replacement:

*tytön* → *tyttö+N+GEN* → *flikka+N+GEN* → *flika*

In other words, the first step is the result of running *tytön* through the analyzer. The second step is the result of automatically replacing the Finnish lexeme with the equivalent Rauma lexeme. The final step would be the result after applying my Rauma generation rules.

Ultimately, I decided that this feature was outside of the scope of this project. By starting with the Rauma lexeme, I would have to essentially rewrite the generator for the entire language before getting to the unique parts. Or, I would have to focus on just a very narrow solution. It was also more fun to implement it the way that I did instead.

When I initially tried implementing the architecture for lexical replacement, I wrote a short Python script to act as a harness (see *fst\_harness.py*). The script uses the HFST Python wrapper library to load the Finnish analyzer, analyze a given surface form, replace lexemes, and feed the result into the Rauma generator. While these steps could have been done natively using 100% FST technology, I thought a Python command line interface (CLI) would be a nice feature.

The intention was also to create certain CLI tool flags which would describe the context of the input word. Some rules create changes at word boundaries depending on the final and following consonants. For instance, a word ending in the letter N drops the consonant if the following word starts with the letters K, T, or P. These environments would have potentially been encoded as an “\_K”, “\_T”, or “\_P” appended to the end of the lexeme, which the Python script would handle. This idea was ultimately abandoned for simplicity’s sake.

Some rules are also dependent on syllable position. I had hypothesized a system that numbered syllables using a Python implementation like so:

1suo2me3a^+N+PART

Such a solution, like the previous CLI flag, would add unnecessary complexity. I ended up opting for a completely finite-state solution which was more elegant. However, I may have been too flexible with my definition of a syllable, which led to extra transformations in places where they should not have occurred (see the internal V ellison in *kivullinen* in the appendix).

Many of the grammar notes had the ill-defined disclaimer of “sometimes.” This disclaimer is rather unhelpful when it comes to writing XFST rules. I decided to deal with these “sometimes” rules by using optionality. Unfortunately, a consequence of using a lot of optionality is that the potential outputs explode, as is very evident in the test data (see the appendix). I still considered each test a victory if the correct form appears somewhere in the potential output space.

## Results

As mentioned above, the appendix contains the system output for each of the test cases I designed. Correct outputs were highlighted in green. Outputs where the rule in question was

successful but some other rule was incorrectly applied were highlighted in yellow. All other outputs are either plausible or incorrect, but I did not sort through all of them to denote the difference.

Due to the complexity of the rule restrictions in some cases, composing all of the rules together leads to an explosively large FST that takes too long to process. The case and particle rules were particularly prone to this explosion. I reckon that it has something to do with checking for a case tag at the end of the sequence to make a change earlier in the sequence. In any case, I ran the final tests in two batches. The first batch composed all rules excluding the case and particle ones and tested only transformations A through R. The second batch composed all the case and particle rules with gemination and last vowel removal and tested only transformations S through X.

In the end, I was able to get a passing test case for each of the target transformations. Some of the case transformations were only partially passing since they depended on one of the other rules not included in that batch. This success is somewhat undermined by all of the optionality. What good is one correct answer when there are three other possibilities? Some of the possibilities are also valid, but the rest are not. Generally, the longer the word is, the more volatile the output. This unreliability makes such a solution intractable for NMT data augmentation as I mentioned in the introduction. Again, all results can be seen in the appendix below.

Perhaps unsurprisingly, language is a very complicated phenomenon to boil down to a set of XFST rules. However, it still seems to be useful for producing a small set of candidates. The explainability of the model and my “expert” knowledge also makes using FST results way more feasible than a rough NMT output with limited interpretability. While it was fun to work on this project, I am not sure that there is a practical use case for what I have created.

## Appendix: Annotated Test Suite

From a test without the case and particle rules:

Test A (Fast Spoken Forms) is omitted for reasons explained above.

### B. End Elision

# kyllä^ -> kyll

kyl

kyll

kyllä

kylä

# jumala^ -> jumal

jumal

jumala

juml

jumla

### C. Internal Elision

# raumalainen^ -> raumlaine

ramalaine

ramalane

ramlaine

ramlane

raumalaine

raumalane

raumlaine

raumlane

# makkara^ -> makkra

makkar

makkara

makkr

makkra

### D. Vowel Shortening

# totuus^ -> totus

tottus

tottuus

totus

totuus

# rehellisyys^+ADJ -> rehellisys

rehelissys

rehelissyys

rehelisys

rehelissyys

rehellissys

rehellissyys

rehellisys

rehellissyys

rehlissys

rehlissyys

rehlisys

rehlissyys

rehllissys

rehllissyys

rehllisys

rehllisyys

E. Diphthong Transformations

# työ mies<sup>^</sup> -> työ miäs

työ miäs

**työ miäs**

# kieli<sup>^</sup> -> kiäl

kiel

kili

**kiäl**

kiäli

# köyhä<sup>^</sup>+ADJ -> kööh

**kööh**

kööhä

F. Internal and Final I

# valkoinen<sup>^</sup>+ADJ -> valkone

valkne

valkoine

**valkone**

# kaunisti<sup>^</sup> -> kauniste

kanis

kaniste

kanisti

kaunis

**kauniste**

kaunisti

G. Consonant Gemination

# ostakaa<sup>^</sup>+V -> ostakka

ostaka

ostakaa

**ostakka**

ostakkaa

# lisää<sup>^</sup> -> lissä

**lissä**

lissää

lisä

lissää

H. Consonant Nasal Assimilation

# sentään<sup>^</sup> -> sendä

**sendä**

sendää

I. Final N

# kodin<sup>^</sup> -> kodi

kod

**kodi**

# vaan<sup>^</sup> -> vaa

va

**vaa**

J. Final T

# pojat^ -> poja

poja

K. Double M, N, R, L

# käännän^+V -> käänä

kän

känn

kännä

känä

kään

käänn

käännä

käänä

# maallinen^+ADJ -> maaline

maaline

maalline

maallne

maalne

maalo

maaloine

maline

malline

mallne

malne

maloi

maloine

L. Common TS

# metsä^ -> mettä

met

mettä

# katso^+V -> katto

kat

katto

M. Common HT

# tapahtuu^+V -> tapattu

tapattu

tapattuu

# humahtaa^+V -> humatta

humatta

humattaa

N. Internal V

# hevonen^ -> heone

hene

heone

hevne

hevone

# savu^ -> sau

sa

sau

sav

savu

### O. Internal J

# marja<sup>^</sup> -> mari

mar

**mari**

# kirja<sup>^</sup> -> kiri

kir

**kiri**

### P. Imperfect Verbs

# istuin<sup>^</sup>+V -> istusi

istus

**istusi**

# nukui<sup>^</sup>+V -> nukusi

nukus

**nukusi**

### Q. Comparative Adjectives

# isompi<sup>^</sup>+ADJ -> isove

is

ise

isov

**isove**

isv

isve

# vanhin<sup>^</sup>+ADJ -> kaikkist vanheve

kaikkist vanhev

**kaikkist vanheve**

vanh

### R. -lisen Adjectives

# ihmellinen<sup>^</sup>+ADJ -> ihmeloine

ihmeline

ihmelline

ihmellne

ihmelne

ihmeloi

**ihmeloine**

ihmline

ihmlline

ihmlne

ihmlne

ihmloi

ihmloine

# kivullinen<sup>^</sup>+ADJ -> kivuloine

kiline

killine

killne

kilne

kiloi

kiloine

kiuline

kiulline

kiullne



kiulne  
kiuloi  
kiuloine  
kivline  
kivlline  
kivllne  
kivlne  
kivloi  
kivloine  
kivuline  
kivulline  
kivullne  
kivulne  
kivuloi  
kivuloine

From a test with only the case and particle rules:

S. Partitive

# ruokaa^+PART -> ruaka

ruak

ruaka

# suomea^+PART -> suami

suam

suami

T. Inessive

# kissa^ -> kiss

kiss

kissa

# talossa^+INE -> talos

talos

# metsässä^+INE -> metäs

mettäs

U. Illative

# kirjaan^+N+ILL -> kirjaha

kirjah

kirjaha

# aitaan^+N+ILL -> aittaha

aitah

aitaha

# punaiseen^+ADJ+ILL -> punasse

punais

punaise

V. Adessive and Allative

# lattialle^+ALL -> laattjall

lattiall

lattialle

# lattialla^+ADE -> laattjall

lattiall

lattialla

# pihamaalle^+ALL -> pihamaall

pihamaall

pihamaalle

# pihamaalla^+ADE -> pihamaall

pihamaall

pihamaalla

W. Comitative

# kahvipannuineen^+COM -> kahvipannuines

kahvipannuines

# pyssyineen^+COM -> pyssyines

pyssyines

X. Clitics

# mitäpä^ -> mitästis

mitästis

# kuulepas^ -> kuulestis

kuulestis